Seminar 02

30.10.2026

Lernziel: Start mit Springboot und Kotlin.

1. Entities und Relationen

Modelliere eine TodoListe:

- `User` schreibt mehrere `TodoList` (`@OneToMany` / `@ManyToOne`)
- Eine `TodoList` erhält mehrere `TodoItem` (`@OneToMany` / `@ManyToOne`)
- Ein `TodoItem` kann mehrere `Tag` haben und umgekehrt (`@ManyToMany` mit `@JoinTable`)

```kotlin
@Entity(name = "todoItem")
@Table(name = "todo_item")
class TodoItem(
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  val id: Long? = null,
  // hier gehts weiter
)
```

Teilaufgaben

1. Implementiere alle vier Entities mit den passenden Annotationen (`@JoinColumn` , `mappedBy` , `@JoinTable`).
2. Warum muss `id: Long? = null` nullable sein?
3. Ergänze `cascade` und `orphanRemoval` bei `TodoList` → `TodoItem`
4. Ergänze Datenfelder mit `@Column` , z.B. für `TodoItem` : `title` (max. 100 Zeichen, nicht null), `done: Boolean` , `dueDate?`

2. Repository & Query-Erstellung

Erstelle ein `TodoItemRepository` mit `JpaRepository` und implementiere:

1. Eine **automatisch generierte Query** über den Methodennamen, z. B. alle offenen (`done = false`) Items einer Liste, sortiert nach Fälligkeitsdatum.
2. Eine **manuelle Query** mit `@Query` für die Suche nach Titel (case-insensitive, alle Listen eines Users).
3. Teste beide Methoden mit einem `@DataJpaTest` (oder direkt in der laufenden Anwendung mit Initialdaten)

3. Service & Transaktionen

Implementiere einen `TodoService` mit:

- `getTodos(userId: Long?, done: Boolean?)` : Filtern über das Repository
- `createTodo(listId: Long, dto: PostTodoItemDto)` : inkl. Mapper (DTO → Entity)
- `toggleDone(todoId: Long)` : Item abhaken
- `deleteList(listId: Long)` : löscht die List und kaskadierend alle Items
- Füge `@Transactional` an die passenden Funktionen an

4. Controller, DTOs & Mapper

Baue einen `TodoController`:

Mapping	Methode	DTO (Request)	DTO (Response)
`GET /todos`	alle Todos (optional `?done=true`)	-	`GetTodoItemDto`
`GET /todos/{id}`	ein Todo	-	`GetTodoItemDto`
`POST /lists/{listId}/todos`	Todo in List anlegen	`PostTodoItemDto`	`GetTodoItemDto`
`PUT /todos/{id}/done`	Todo abhaken	-	`GetTodoItemDto`
`DELETE /todos/{id}`	Todo löschen	-	-

Teilaufgaben:

- Nutze `@PathVariable`, `@RequestParam` und das passende Mapping pro Methode.
- Setze `@ResponseStatus` korrekt (OK, CREATED, NO_CONTENT).
- Schreibe die Mapper-Funktionen `Entity` → `GetTodoItemDto` und `PostTodoItemDto` → `Entity`. Warum enthält `PostTodoItemDto` keine `id`?
- Teste alle Endpunkte.